

```

pullDownDelay)
    }
    // Delay between this move and the next one
    val nextMoveDelay = (MIN_MOVE_DELAY_MS..
MAX_MOVE_DELAY_MS).random()
    gameHandler.postDelayed(pullUpRunnable, nextMoveDelay)
    } } }
-----
-----

```

Remember that the sample code works to track the position of the robot and make data points of the movement. Not all the droids will return an execution result on being hit which is why a default case must be specified as to how interactive systems must respond.

```

private fun failHit() {
    scoreboard.score -= 50
    scoreboard.life -= 1
    failLight?.blink()
    if (scoreboard.life <= 0) {
        // Game over
        gameHandler.removeCallbacksAndMessages(null)
        grid.flatMap { it.toList() }
            .filterNotNull()
            .filter { it.position != DOWN && it.position
!= MOVING_DOWN }
            .forEach { it.pullDown() }
    }
}
-----
-----

```

The final code is a simple update function to update the position of the sample droids and to keep them in check if they don't respond. The code is used to report any hits or misses when the users interact with the droids once they appear in the device.

```

this.setOnTapListener { _, _ ->
    if (this.position != DOWN) {
        // Droid hit! assign 100 points
    }
}

```